

#UNIT-1

#NLP-

- Natural Language Processing

- Branch of AI that provides computers with the capability of understanding text & spoken words in the same way a human being can

- Combines linguistics & computer science to analyze language structure

- Abilities:

(a) Read

(d) Speak

(b) Understand

(e) Listen

(c) Interpret

- Applications:

- Language translation

- Smart assistant

- Document analysis

- Online searches

- Sentiment analysis

- Phases:

(i) Lexical analysis

(v) Pragmatic analysis

(ii) Syntactic analysis

(iii) Semantic analysis

(iv) Discourse integration

Phases of NLP -

1) Lexical analysis / morphological processing:

- Deals with study of trying to understand the meaning of words, & their relation
- In this phase, the sentences, paragraphs are broken into tokens
← Smallest unit of text

- Types:

- Tokenization: sequence of normalized tokens
- Recognition: whether sequence of word is valid
- Morphological Parsing: analyze the structure of the word
- Morphological Generation: generates words from root & grammar

2) Syntactic analysis / parsing:

- Sentences are checked whether it is well formed or not
← "Delhi goes to him"
X

3) Semantic analysis:

- Sentences are checked for literal meaning of each word & their arrangement together
X "I ate ice cream"

4) Discourse integration:

- In this phase, impact of the sentences before a particular sentence & the effect of the current sentence on the upcoming sentences is determined

eg- (i) Riya brought a laptop } She = Riya
(ii) She loves it } it = laptop

5) Pragmatic analysis:

- Actual effect of the text is discovered by applying the set of rules that characterize cooperative dialogues
- Understands the speaker's real intention using context

eg- (i) "close the window?" should be interpreted as a request instead of an order

(ii) "Nice job!" after you accidentally deleted a project

Literal: Compliment

Actual: Sarcasm

Only context reveals this

Regular Expression -

- Sequence of characters that specified a search pattern in text.
- Used to find, find & replace, text extraction and input validation operations

Text: contact us at support@company.com
or call 9876543210

Regex for phone number: `\d{10}`

Output: 9876543210

- Common regex symbols:

(i) `.` = Any single character $\Rightarrow$ `ct` $\rightarrow$ `cat`, `cot`

(ii) `*` = Zero or more occurrences $\Rightarrow$ `ab*` $\rightarrow$ `a`, `ab`, `abb`

(iii) `+` = One or more occurrences $\Rightarrow$ `ab+` $\rightarrow$ `ab`, `abb`

(iv) `?` = Zero or one occurrence $\Rightarrow$ `colou?` $\rightarrow$ `color`, `colour`

(v) `^` = Start of string $\Rightarrow$ `^Hello`

(vi) `$` = End of string $\Rightarrow$ `world$`

(vii) `[]` = Character set $\Rightarrow$ `[aeiou]`

- Common use of regex in NLP -

- (i) Tokenization
- (ii) Information extraction
- (iii) Text cleaning
- (iv) Pattern matching
- (v) Data validation

Edit Distance Regular Expression -

- Used to measure the similarity between 2 strings by calculating the minimum number of operations required to transform one string into another
- Most commonly used edit distance is the Levenshtein distance

+ Insertion

+ Deletion

+ Substitution (Replacement)

- Formula: For 2 strings A and B

Edit distance (A, B) = Minimum number of insertions, deletions & substitutions required to convert A into B

eg - Convert Sunday $\rightarrow$ Saturday
Edit distance = 3

Operation	Result
Insert A $\rightarrow$	SAUNDAY
Insert T $\rightarrow$	SATUNDAY
Replace N with R $\rightarrow$	SATURDAY

- Application of Edit distance:

- Spell checking
- Auto correction
- Machine Translation Evaluation

Text Granularity in NLP -

Corpus (large collection of documents)

$\downarrow$

Paragraph

$\downarrow$

Sentence $\rightarrow$ Word

NLP step -

(i) Sentence segmentation

(ii) Tokenizing

(iii) Removing stop words

(iv) Stemming

(v) Lemmatization

(vi) Part of speech tagging

(vii) Named entity tagging

Sentence Segmentation -

Process of dividing a large block of text into individual sentences.

Tokenization -

Process of breaking a sentence into smaller units called tokens

Removing stop words -

Remove common words

Normalization -

Process of converting text into a standard and consistent format

Stemming -

- Process of finding root of words & reducing words to their word stems

- Stemmer:

(i) Porter: best for learning stemming concepts

(ii) Regexp: best when you need to remove specific patterns

(iii) Snowball: more accurate, efficient & best

leaf → leab
leaves
→ leav

→ equal to or smaller form

- Disadvantages:

- Overstemming
- Understemming
- Language challenges

Lemmatization -

- Advanced & more accurate text normalization technique than stemming
- Analyzes the meaning & part of speech (POS) of a word to convert it into its dictionary base form (lemma)

POS Tagging -

Called sequence labelling because one word can depend on prev &/or surrounding words

- Process of assigning a grammatical category (noun, adjective, adverb, verb, etc) to each word ~~of~~ in a sentence based on its meaning & context

- Types:

- Noun (N)
- Verb (V)
- Adjective (ADJ)
- Adverb (ADV)
- Preposition (P)

- Conjunction (CON)
- Pronoun (PRO)
- Interjection (INT)

Named Entity Tagging -

- Also called Named Entity Recognition
- Process of identifying & classifying important real world entities in text into predefined categories

Location	-	Person
Date	-	Organization
Money	-	Time
	-	Percentage

Introduction to word embeddings -

- Method of converting words into dense numerical vectors to ensure semantic relationships.
- Similar words are placed closer together in the vector space

- Steps:

(i) Tokenization

(ii) Word Embedding representation

(each word is converted into a dense numerical vector by a word embedding model)

Key Properties: Word embeddings -

(a) Semantic similarity:

Similar words have similar vectors

eg: car & automobile
doctor & physician

(b) Context awareness:

Words appearing in similar contexts tend to have similar embeddings

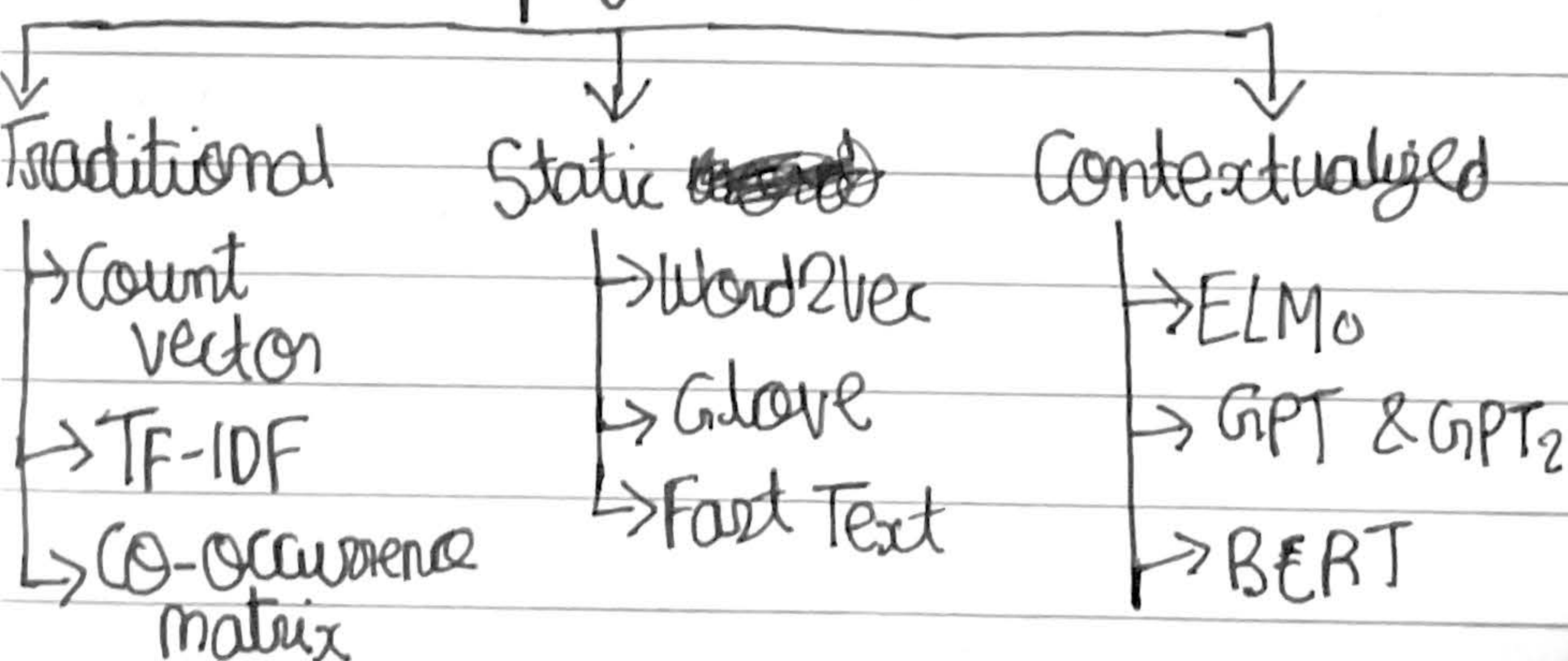
eg- The cat drank milk } cat & dog
The dog drank milk } are similar

(c) ~~Similar~~ Dense representation:

Each word is represented by a dense vector containing real numbers

These values are learned automatically during model training

Word embedding techniques -



Traditional word embedding (Count based) -

- Represent words using counts or frequencies of their occurrences in documents

- Working principle:

(i) Build a vocabulary from the corpus

(ii) Count word occurrences or co-occurrences

(iii) Convert the counts into numerical vectors

(iv) Use these vectors for machine learning tasks

Types of Count based method -

A. One hot encoding:

- Each word is represented by a binary vector

- Vector has a 1 in the position corresponding to the word's index & 0 elsewhere

- Steps:

(i) Tokenization

(ii) Vocabulary Creation

(iii) Vector initialization

(iv) Encoding

- Limitations:

(i) Huge vectors

(ii) No semantic meaning

(iii) Every word is equally distant

B. Bag of Words (BoW):

- Counts how many times each word appears.
- Treats each document as a bag/collection then count the frequency of each word in the document
- Limitations:
 - (i) Ignores word order
 - (ii) No understanding of meaning

C. TF-IDF:

- Improved version of BoW that assigns weights to words based on their importance in a document

(a) Term Frequency (TF) = measures how often a word appears in a document. Higher $\rightarrow$ higher frequency importance

(b) Inverse Document Frequency (IDF) = Measures how rare a word is across all documents.

Rare words get higher weight, while common words get lower weights

$$TF-IDF = TF \times IDF$$

* Frequently occurring words in a document but rare across the corpus receive the highest TF-IDF scores

$$TF(t, d) = \frac{\text{No. of occurrences of term } t \text{ in document } d}{\text{Total no. of terms in document } d}$$

$$IDF(t, d) = \log_e \frac{\text{Total no. of documents in the corpus}}{\text{No. of documents with term } t \text{ in them}}$$

$$TF-IDF(t, d, D) = TF(t, d) \times IDF(t, d)$$

D. Co-occurrence Matrix:

- A table that records how frequently pairs of words occur together in a given context

#Count based methods —

Advantages —:

- + Easy to understand
- + Computationally inexpensive
- + Good for small datasets
- + Works well for traditional machine learning algorithms

Limitations:

- + Cannot understand context
- + Cannot capture word meaning
- + Produces sparse vectors

- ## # Static Word Embedding (Prediction based) -
- Use neural networks to learn word vectors automatically
 - Methods predict words from their surrounding context

A-Word2Vec (Google):

- Generates dense vector representations of words

- Train a neural network to predict context words given a target word

- 2 approaches:

X { Continuous Bag of Words (CBOW) given a its surrounding context, predict target word
Skip-gram

↳ predicts the surrounding context words given a target word

Advantages:

- Faster training
- Works well for freq. words
- Suitable for large datasets

Limitation:

- Poor performance on rare words
- Not detecting Polysemy

B. GloVe (Global Vectors):

- Global vectors for Word Representation
- Unsupervised word embedding technique
- Learn word vectors by analyzing how frequently words co occur throughout the entire corpus

Advantages:

- Captures global word relationships
- Produce dense embeddings
- Efficient on large corpora
- Pre trained vectors are highly available
- Better semantic understanding

Limitations:

- Generates one vector per word
- Cannot distinguish meanings of the same word

Contextual Word Embedding (Transformer based) -

- Generate different vectors for the same word depending on the sentence in which it appears

A. BERT, GPT:

- BERT creates contextual embeddings by looking at both previous & next words
- GPT creates contextual embeddings by looking at previous words only

Root = main word

Dependency Parse Tree -

A dependency graph becomes a dependency parse tree when it satisfies the following properties:

Every dependency consists of 2 words:

└ Head → Important word

└ Dependent → the word that depends on the head

Property 1: Root has no incoming arc.

Root does not depend on any other word

Property 2: Every other word has exactly 1 parent

Property 3: Unique path from root

(a) Projective Parse Tree: A dependency tree in which all dependency arcs are projective

(b) Non Projective Parse Tree: A dependency tree containing at least one non projective arc.